

CSI 2110
Data Structures and Algorithms

Lecture 0: Introduction

Bo Sun

Fall 2026

About this course

- ITI 1121 (or equivalent) is about how to program:
  - translating a problem into codes

- CSI 2110 is about how to program efficiently:
  - choosing appropriate data structures and algorithms to solve a problem
  - explaining mathematically why the code runs efficiently

Why learn about data structures and algorithms?

- Fundamental building block for computer science
  - theory direction: Design and analysis of algorithms
  - systems direction: Databases; Operating systems
- Job interviews
- In the era of generative AI

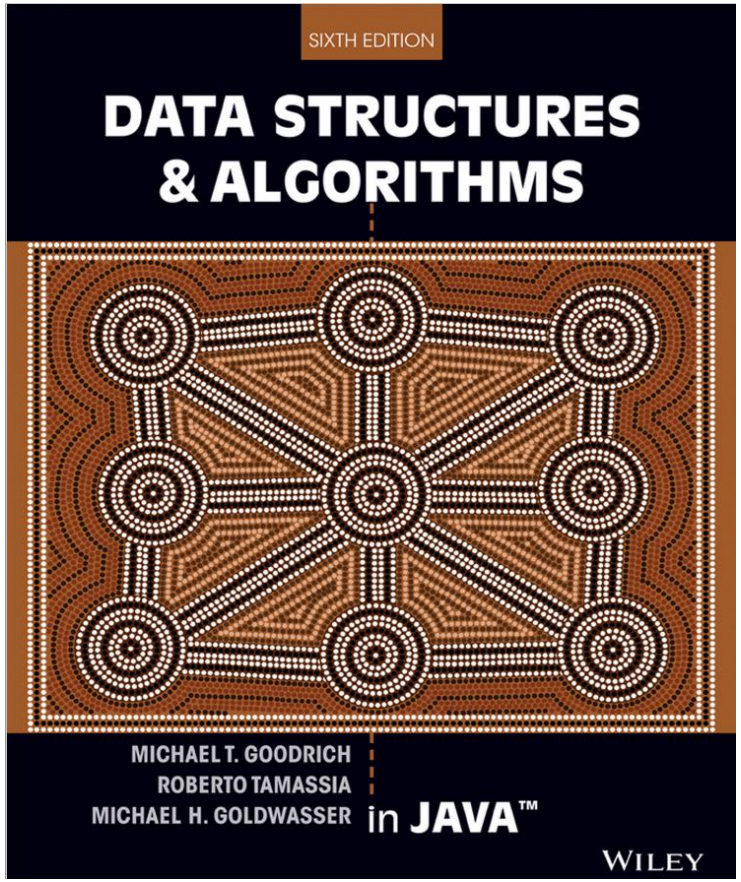
Agenda

- What are data structures?
- Course logistics
- Review of array and linked list

A note on the slides

Adapted from the slides that accompany this book.

Part of the slides is based on previous slides from Prof. Lucia Moura for CSI 2110.



Data Structures and Algorithms in Java
6th edition
Michael Goodrich, Roberto Tamassia, Michael Goldwasser
Wiley, 2014

About the instructor – Bo Sun

- I do research on algorithms
  - specifically, online algorithms, learning-augmented algorithms, and their applications in networked systems
- I got my Ph.D. from the Hong Kong University of Science and Technology (HKUST)
- I joined uOttawa as an assistant professor in July 2025
- Feel free to reach out if you want to talk about research!

Today's agenda

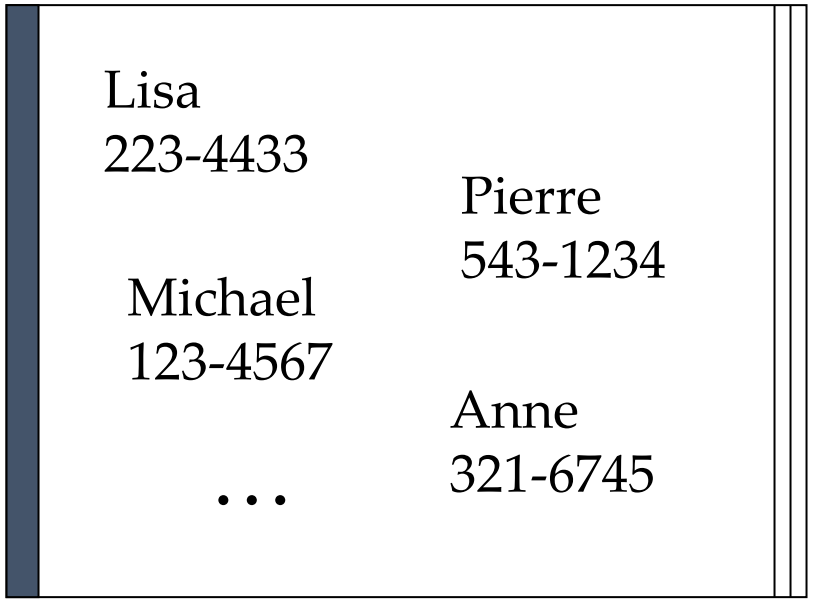
- What are data structures?
- Course logistics
- Review of array and linked list

What is a data structure?

- Data structure is a way of storing and organizing data such that it can be used efficiently
  - storing a set of elements
  - supporting certain operations on those elements

Example: a phone book

- contains data
  - names
  - phone numbers
- needs to perform certain operations
  - add
  - delete
  - search for a phone number



How data is organized logically matters

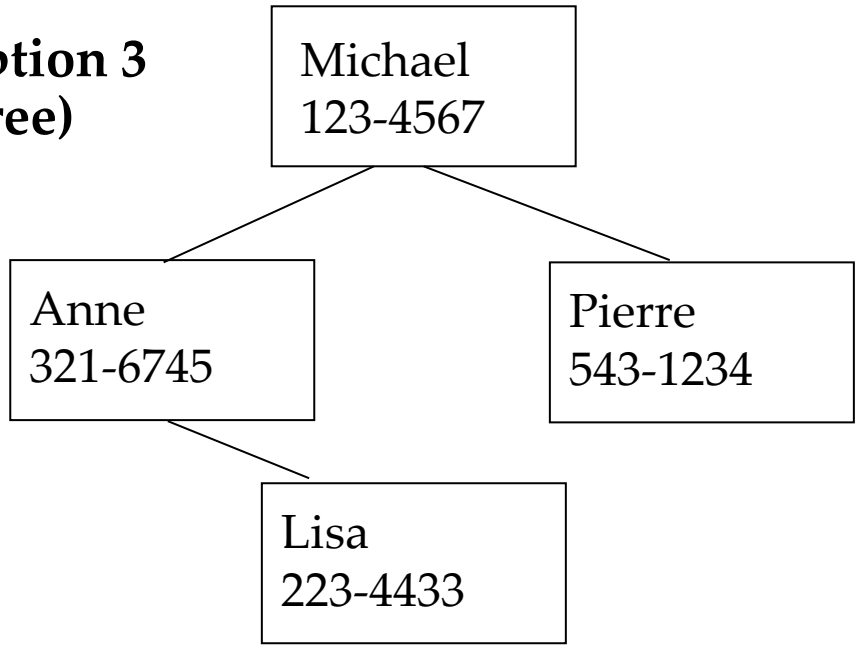
Option 1 (List)

Lisa 223-4433	Pierre 543-1234	Michael 123-4567	Anne 321-6745

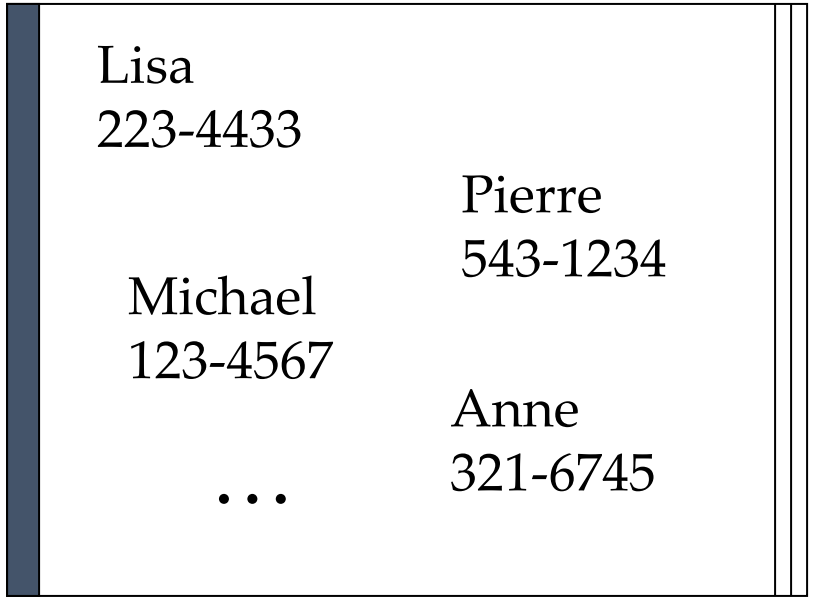
Option 2 (Ordered list)

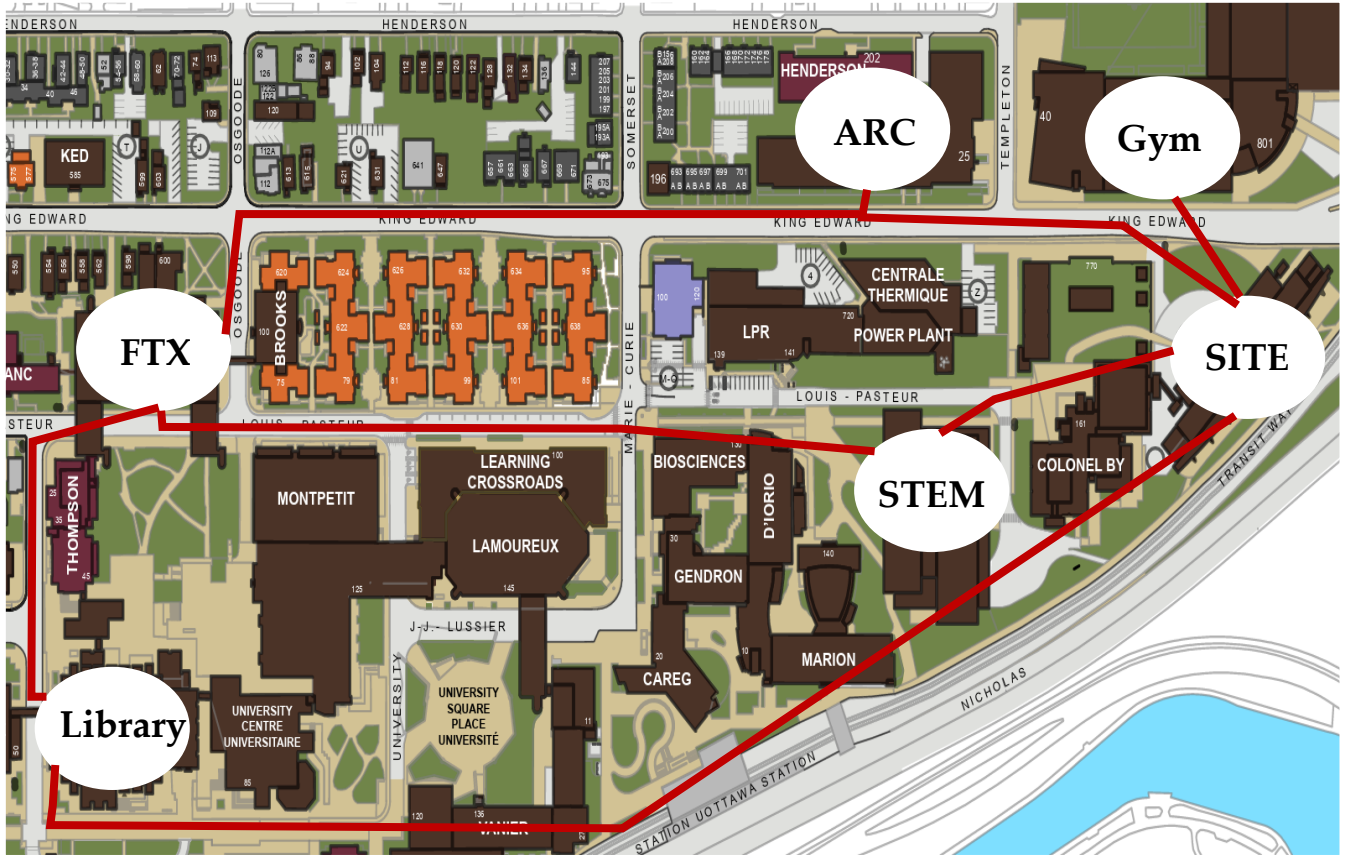
Anne 321-6745	Lisa 223-4433	Michael 123-4567	Pierre 543-1234

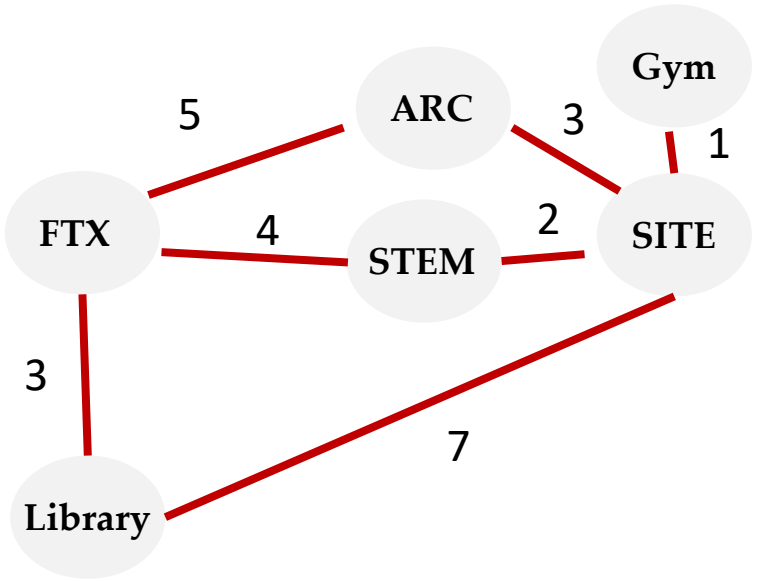
Option 3 (Tree)



Phone book





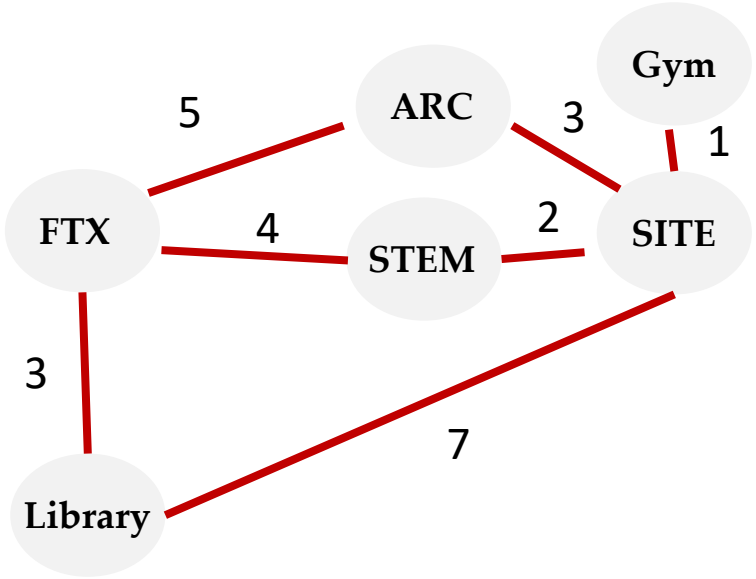


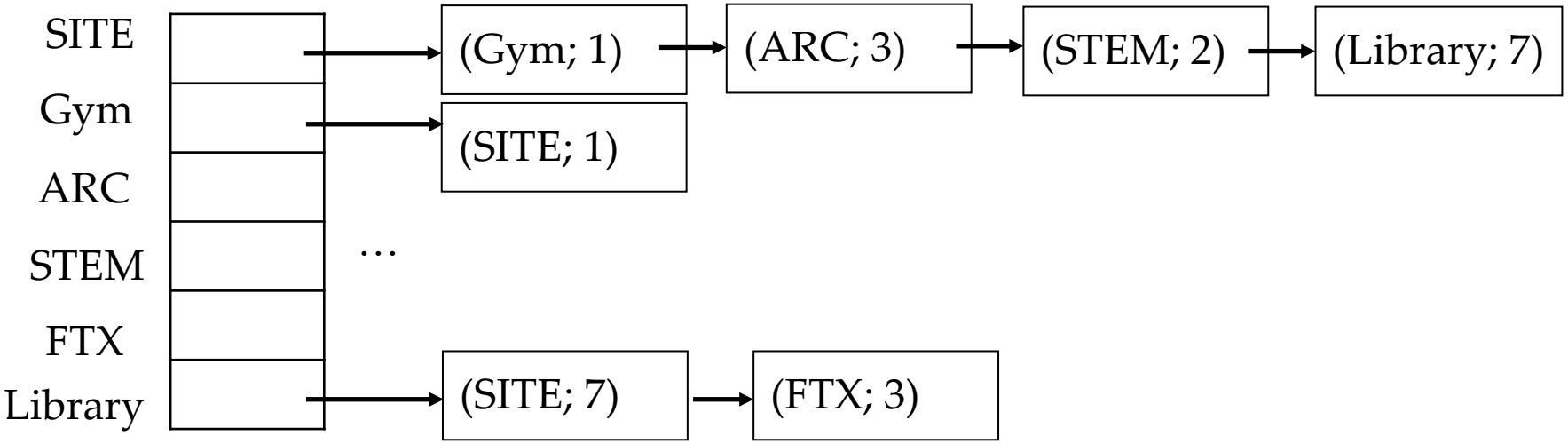
Example: a personal map that

- contains data
  - buildings
  - distances
- needs to perform certain operations
  - e.g., find the shortest route from SITE to Library

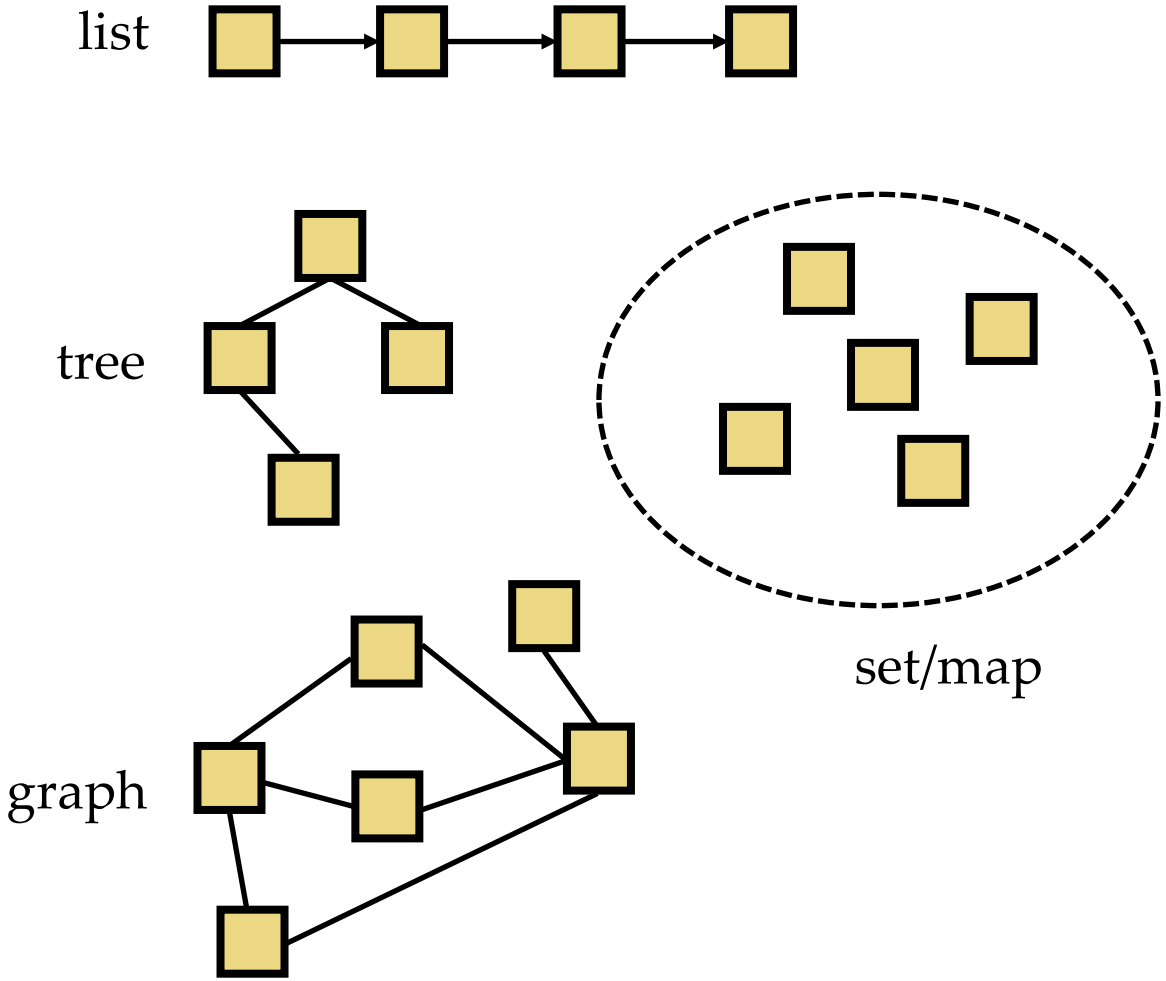
How data is stored physically matters

	SITE	Gym	ARC	STEM	FTX	Library
SITE		1	3	2		7
Gym	1					
ARC	3				5	
STEM	2				4	
FTX			5	4		3
Library	7				3	

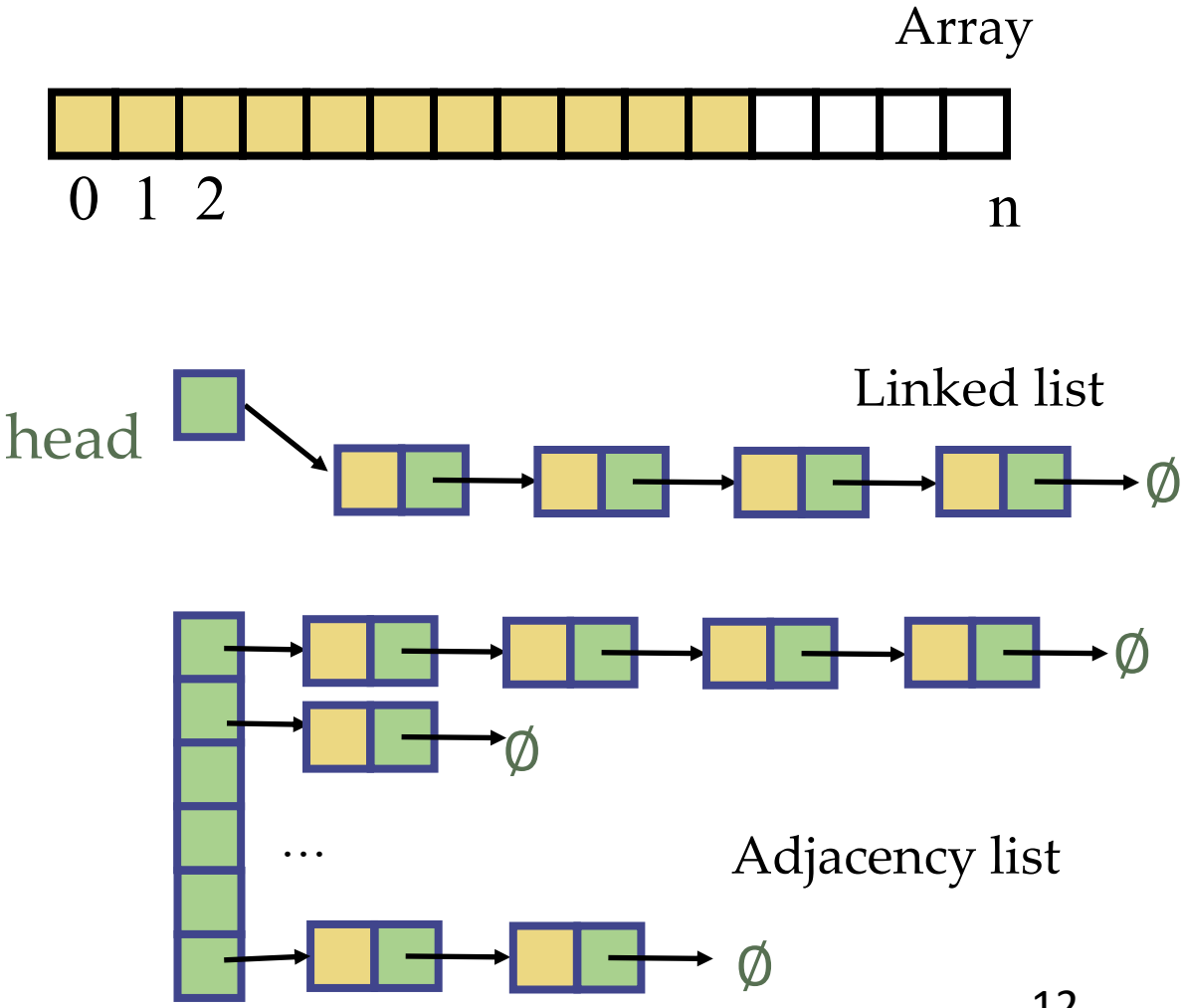




Logical structures



Physical structures (implementation)



Objectives of this course

- know the most commonly used data structures, emphasizing their abstract data properties
- know typical algorithms that operate on each type of data structure, and can analyze their performance
- compare different data structures for solving the same problem, and choose the best
- also, implement data structures and algorithms using Java

Questions?

Agenda

- What are data structures?
- Course logistics
- Review of array and linked list

Course logistics – where to get what ☺

- The course syllabus
  - Link
  - Course outline, tentative schedule, policies

- Brightspace
  - Announcements
  - Lecture notes
  - Assignments
  - Labs
  - Tutorials

Course logistics – Lectures and office hours

- Each week we have:
  - 2 lectures (prof) - theory: data structures, algorithm pseudocode, and analysis of algorithms
  - 1 lab (TA) - practical work in Java: prepares you for programming assignments and reinforces theory study
  - 1 tutorial (TA) - written exercises: prepares you for written assignments and exams
- Instructor office hours are generally for questions about the lectures and other topics related to data structures and algorithms
  - 3pm - 4pm on Tuesdays in SITE 4021
  - 2pm - 3pm on Thursdays in SITE 4021
- TA office hours are for questions about labs, tutorials and assignments
  - Time and location are on Brightspace (General information -> Office hours)

Course logistics – Assessment

	Weight	**Time**
Written Assignments (A)	15%	2 before and 2 after midterm
Programming Assignments (P)	15%	P1 October; P2 November
Midterm (M)	30%	9:30am-11:30am on November 7
Final exam (F)	40%	

- Bonus from Labs (B): 2% for lab participation and evaluation
- Marking scheme:

  If (M + F)/70 >= 50%, then **G** = A + P + M + F + B

  Else **G** = (M + F)*100/70

Course logistics – Written and Programming Assignments

- Assignment specifications will be posted on Brightspace.
- Assignments are to be completed individually and submitted to the appropriate folder on Brightspace.
  - We do not accept assignments via emails
- Late submissions within 1min - 24 hours will be accepted with 30% off. No assignments will be accepted after 24 hours late.

Course logistics – Exams

- Closed-book written exams.

- A minimum of 50% in the weighted grades of midterm and final exam are required to pass the course.

- If you miss the midterm due to specific justifiable reasons (e.g., sickness with doctor's note), the weight of midterm will be shifted to the final exam, and you need to score at least 50% in the final exam to pass the course.

Course logistics – Labs for Session B

- The purpose of the labs is to learn, and put the concepts of the course into practice. All labs in the term together contribute to 2% bonus points.

- The lab sessions (Lab B01 on Mon and Lab B02 on Tue)
  - Release: Lab specifications are posted on Brightspace at 4PM on Thursdays
    - You have one week to work on each lab
  - Submission deadline: 11:59PM on the following Thursday
  - In-person grading: Each student must have their lab graded by a TA during scheduled lab sessions within two weeks after the submission deadline.
  - Late submissions within 1min - 24 hours will be accepted with 30% off. No assignments will be accepted after 24 hours late.
  - More detailed instructions on Brightspace

Course logistics – Labs for Session B

- The first lab session will be
  - released today
  - on Sept 14 for Lab B01 and on Sept 15 for Lab B02
  - due on Sept 17
  - graded in persons on Sept 21&22 or Sept 28&29

- No tutorial next week. The first tutorial is on Sept 21.

Course logistics – Labs for Session C

- The purpose of the labs is to learn, and put the concepts of the course into practice. All labs in the term together contribute to 2% bonus points.

- The lab sessions (Lab C01&C04 on Fri and Lab C02 on Mon)
  - Release: Lab specifications are posted on Brightspace at 4PM on Thursdays
    - You have one week to work on each lab
  - Submission deadline: 11:59PM on the following Thursday
  - In-person grading: Each student must have their lab graded by a TA during a scheduled lab sessions within two weeks after the submission deadline.
  - Late submissions within 1min - 24 hours will be accepted with 30% off. No assignments will be accepted after 24 hours late.
  - More detailed instructions on Brightspace

Course logistics – Labs for Session C

- The first lab session will be
  - released today
  - on Sept 11 for Lab C01&C04 and on Sept 14 for Lab C02
  - due on Sept 17
  - graded in persons on Sept 18&21 or Sept 25&28

- No tutorial next week. The first tutorial is on Sept 21.

Course logistics – Q&A Policy

- We strongly encourage you to ask questions!

- We will do our best to answer your questions as soon as possible, but
  - it may take the teaching team up to 2 business days to answer emails.

- Please take that into account when planning when to start working on assignments and studying for the exams.

Course logistics – Academic Integrity

- The person copying and the person providing the material for the copying will both be considered as Plagiarism.
- The use of generative AI tools such as ChatGPT in any submission is considered as academic fraud.
- If you choose to use any code from the Internet (e.g., GeeksforGeeks), you must clearly include the references in the report. This may lead to a deduction in your grade.
- Please read Academic Integrity for Students.

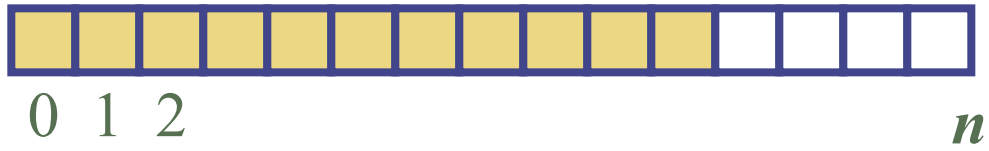
Questions?

Agenda

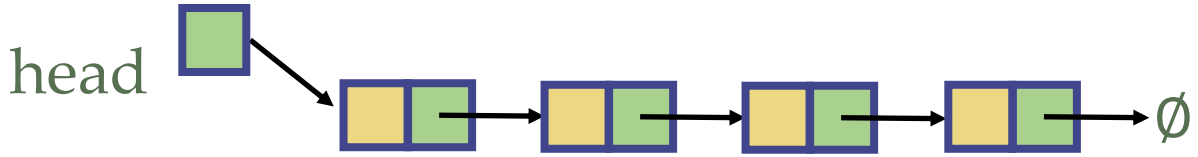
- What are data structures?
- Course logistics
- Review of array and linked list

Review of fundamental data structures

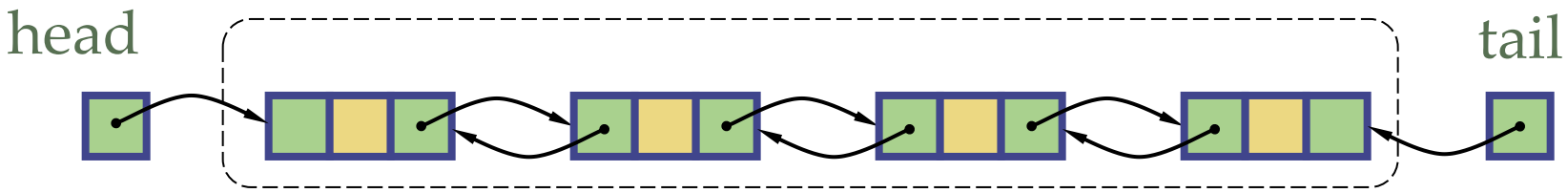
- Array



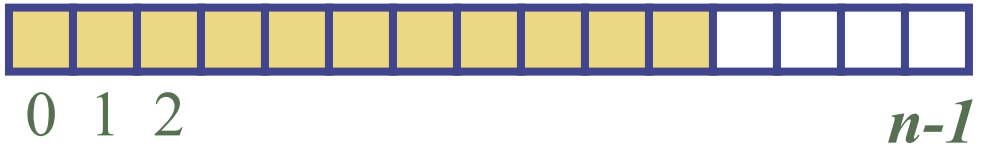
- Singly Linked List



- Doubly Linked List

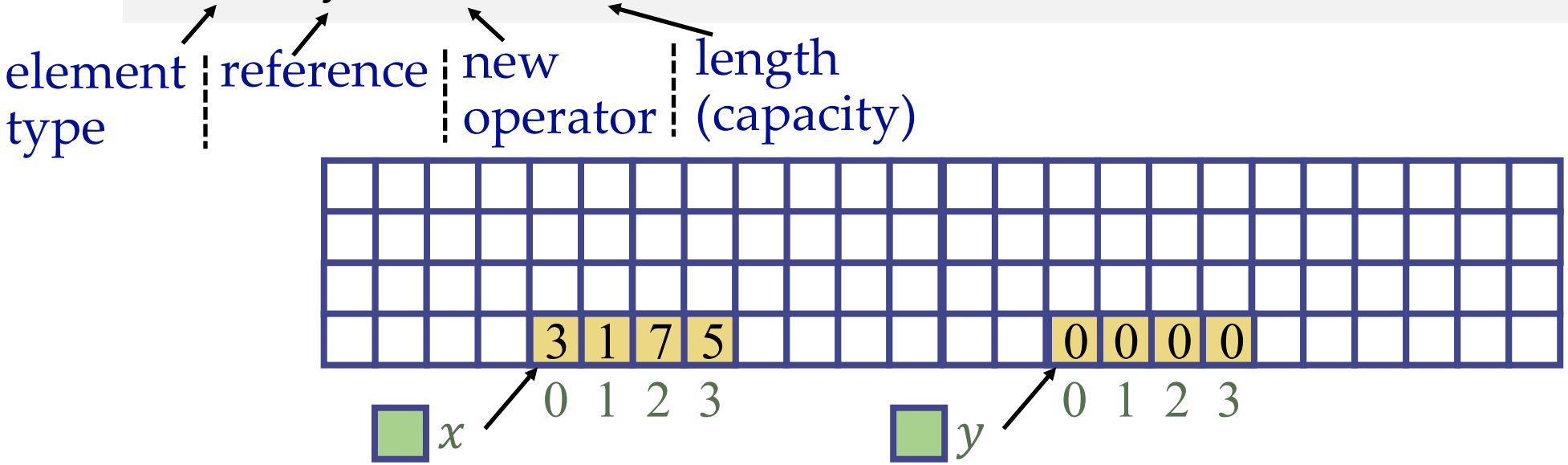


Array



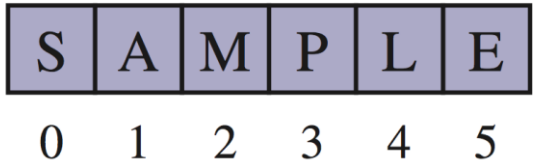
- Array is a sequence of elements, all of the same type
  - fixed length (capacity)
  - each element can be accessed directly
- Declaring an array in Java:

```
int[] x = {3, 1, 7, 5};        ▷ declaring an integer array with initiation
int[] y = new int[4];          ▷ declaring an empty integer array
```

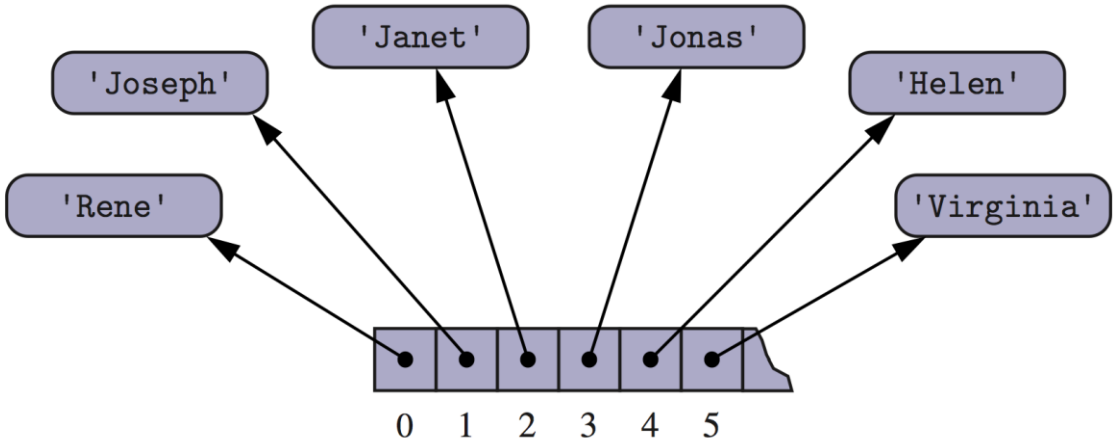


Arrays of characters or object references

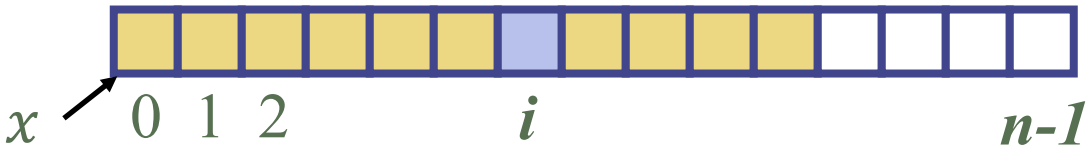
- An array can store primitive elements, such as int, double characters, etc.



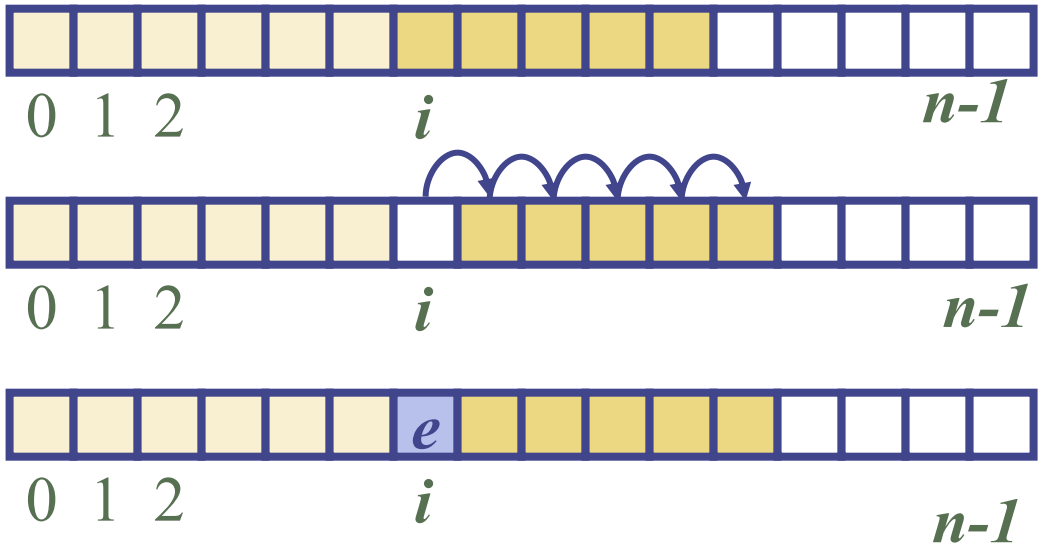
- An array can store references to objects



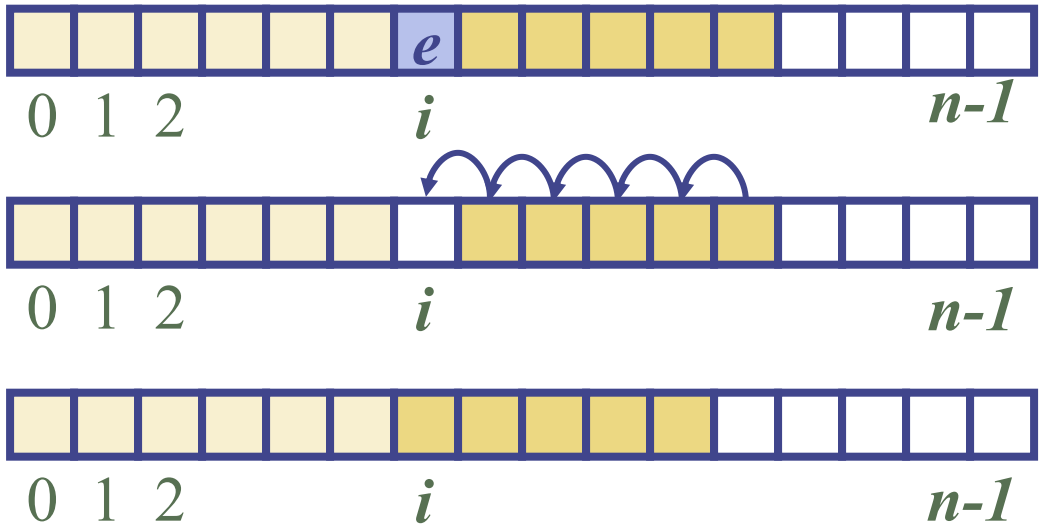
Operations on arrays



- Random access at index i at a constant time
- Inserting at index i



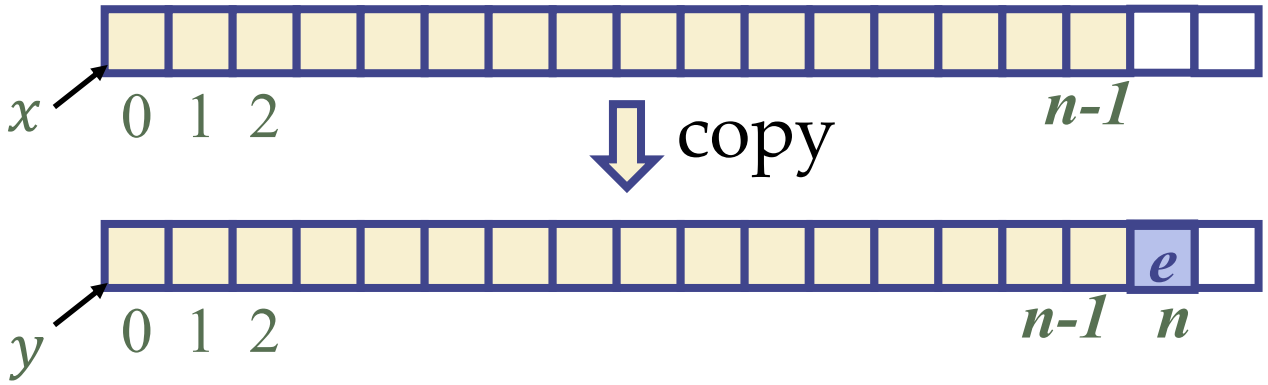
- deleting an element at index i



For insertions and deletions at a specific position, other entries must be moved. Memory copy is time consuming!

Operations on arrays

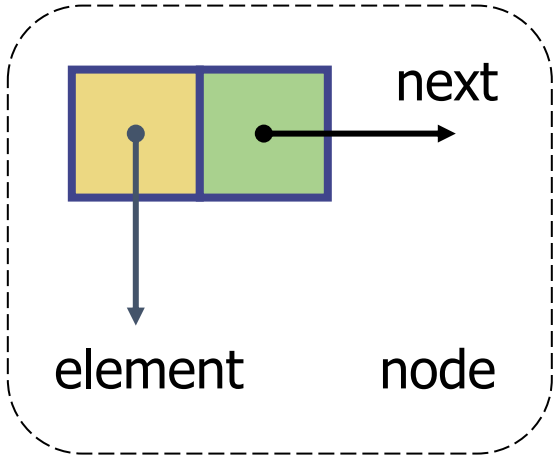
- Inserting an element *e* into a full array

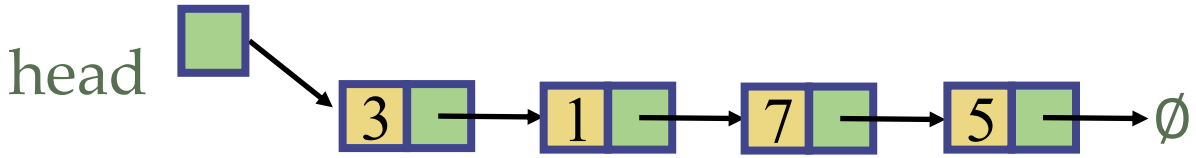


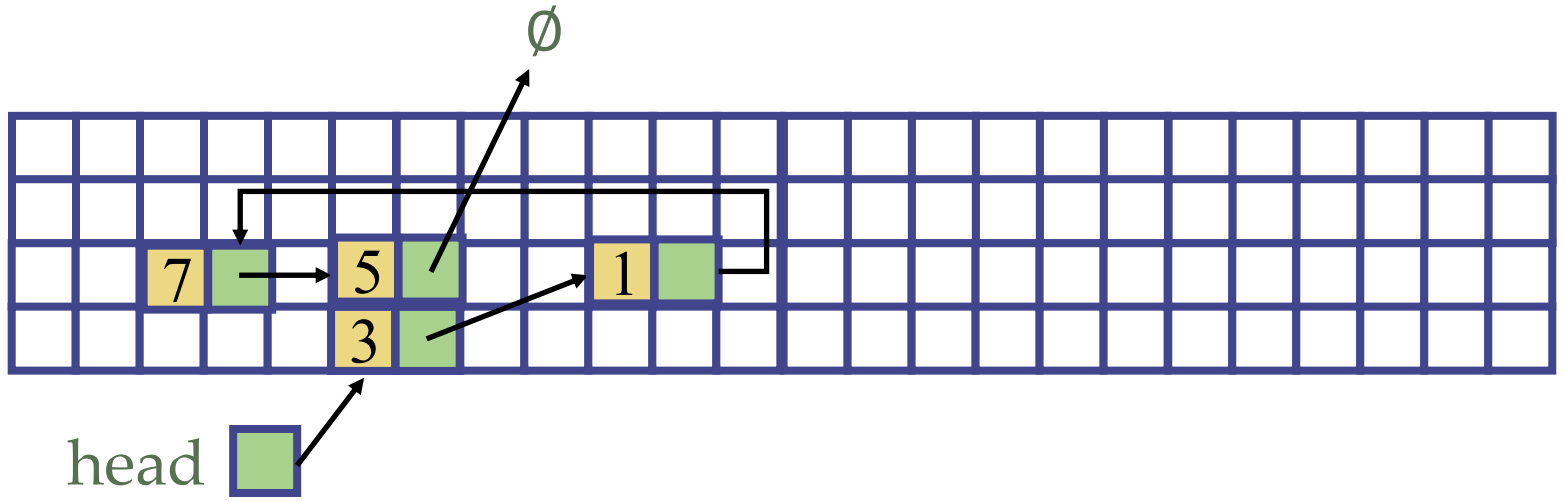
An array has a fixed size. Adding elements beyond its capacity needs to create a new array and copy the existing elements.

Singly linked list (SLList)

- A singly linked list is a sequence of nodes, where
  - each node stores an element and a link to the next node
  - a head pointer stores the link to the first node





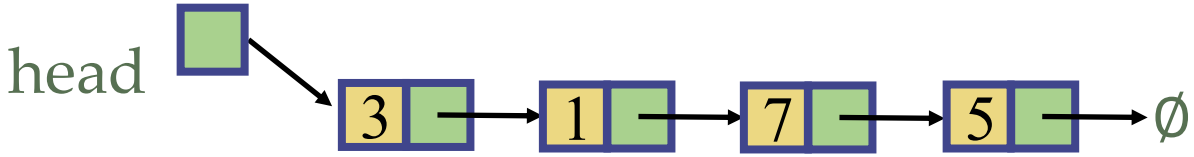


```java
public class SLList {
    /** A nested node class */
    public Node head;

    public SLList(int e) {
        head = new Node(e, null);
    }

    public void addLast(int e) {
        Node current = head;
        while(current.next != null) {
            current = current.next;
        }
        current.next = new Node(e, null);
    }

    public void print() {
        Node current  = head;
        while(current != null) {
            System.out.print(current.element);
            current = current.next;
        }
        System.out.println();
    }
    /** create and print the link list */
}
```



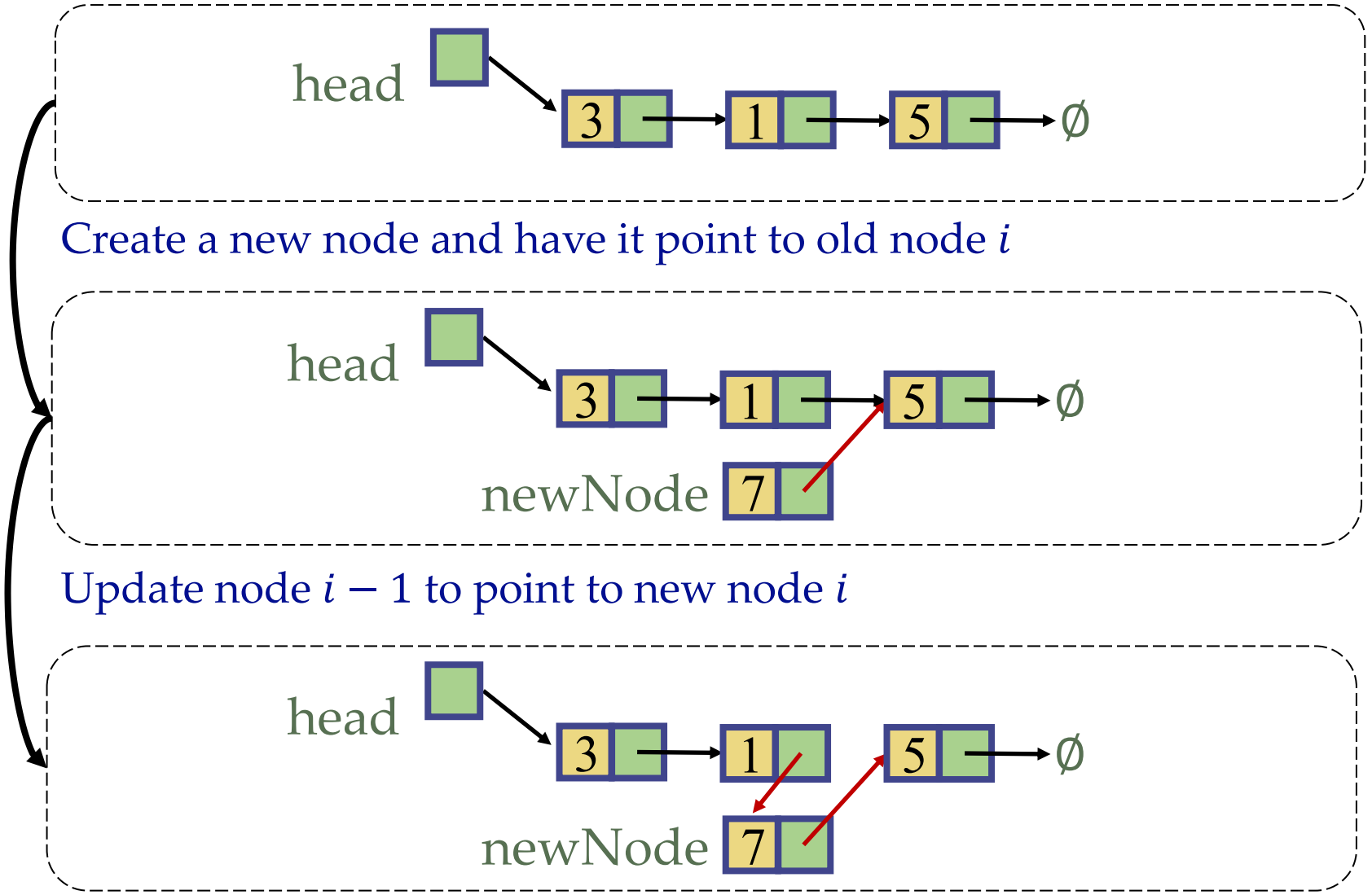

```java
    /** A nested node class */
    public class Node {
        public int element;
        public Node next;

        public Node(int e, Node n) {
            element = e;
            next = n;
        }
    }
```

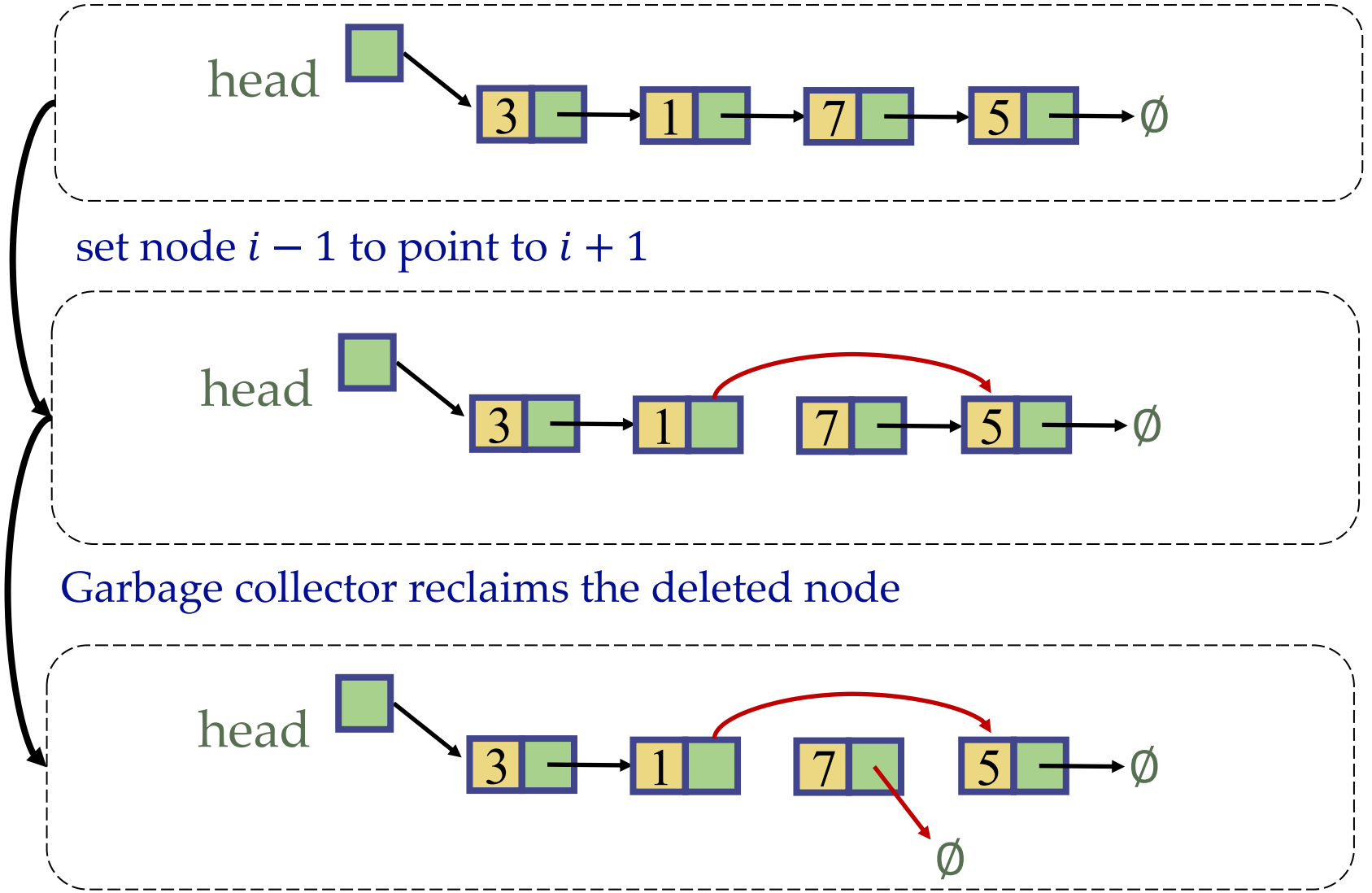
```java
 /** create and print the link list */
 public static void main(String[] args) {
        SLList list = new SLList(3);
        list.addLast(1);
        list.addLast(7);
        list.addLast(5);

        list.print();
    }
```

Operations on SLList: Inserting an element at node *i*



Operations on SLList: Deleting the element at node *i*



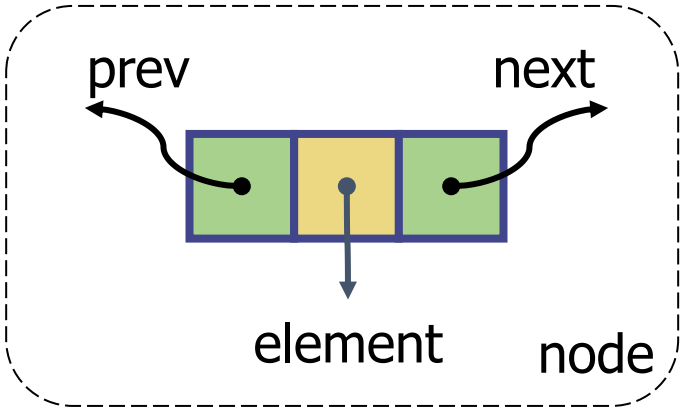
Operations on SLList

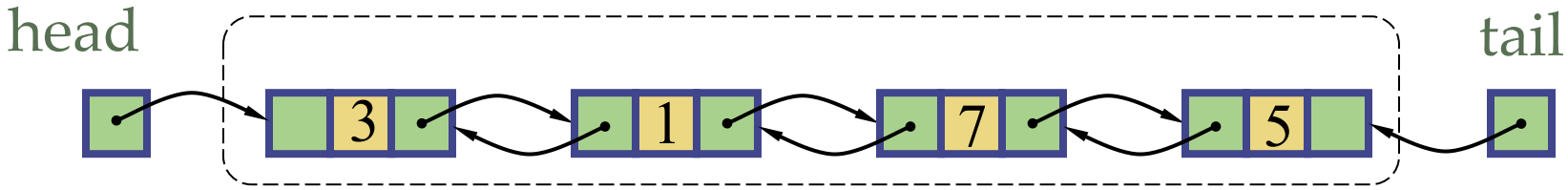
- Inserting or deleting the last node is slow (why)
- Suppose we want to quickly insert or delete the last node (i.e., tail node), will it be sufficient to store a tail pointer?
  - Yes for inserting
  - **No for deleting (why)**

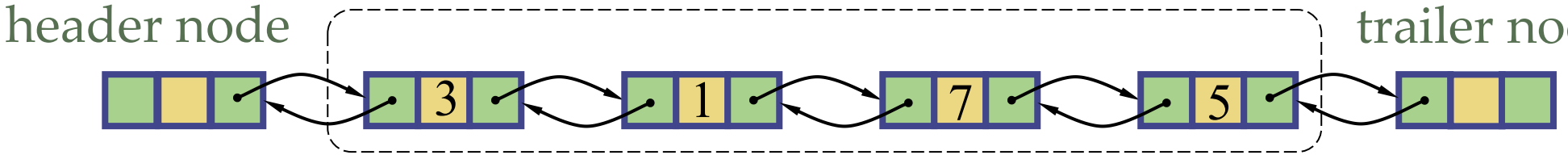
Deleting the last element needs to traverse the list, even if a tail pointer is stored

Doubly linked list (DLList)

- A doubly linked list is a sequence of nodes, where
  - each node stores an element, a link to the next node, and a link to the previous node
  - a head pointer and a tail pointer





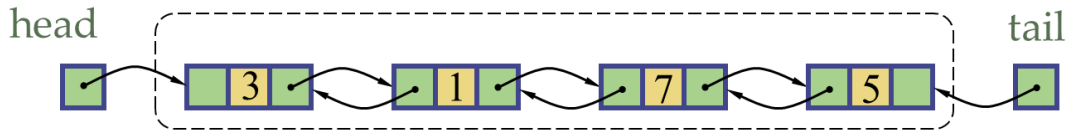


```java
public class DLList {
    /** a nested node class */
    public Node head;
    public Node tail;

    public DLList(int e) {
        head = tail = new Node(null, e, null);
    }

    public void addLast(int e) {
        tail.next = new Node(tail, e, null);
        tail = tail.next;
    }

    public void print() {
        Node p = head;
        while(p != null) {
            System.out.print(p.element + " ");
            p = p.next;
        }
        System.out.println();
    }
    /**create and print the link list */
}
```



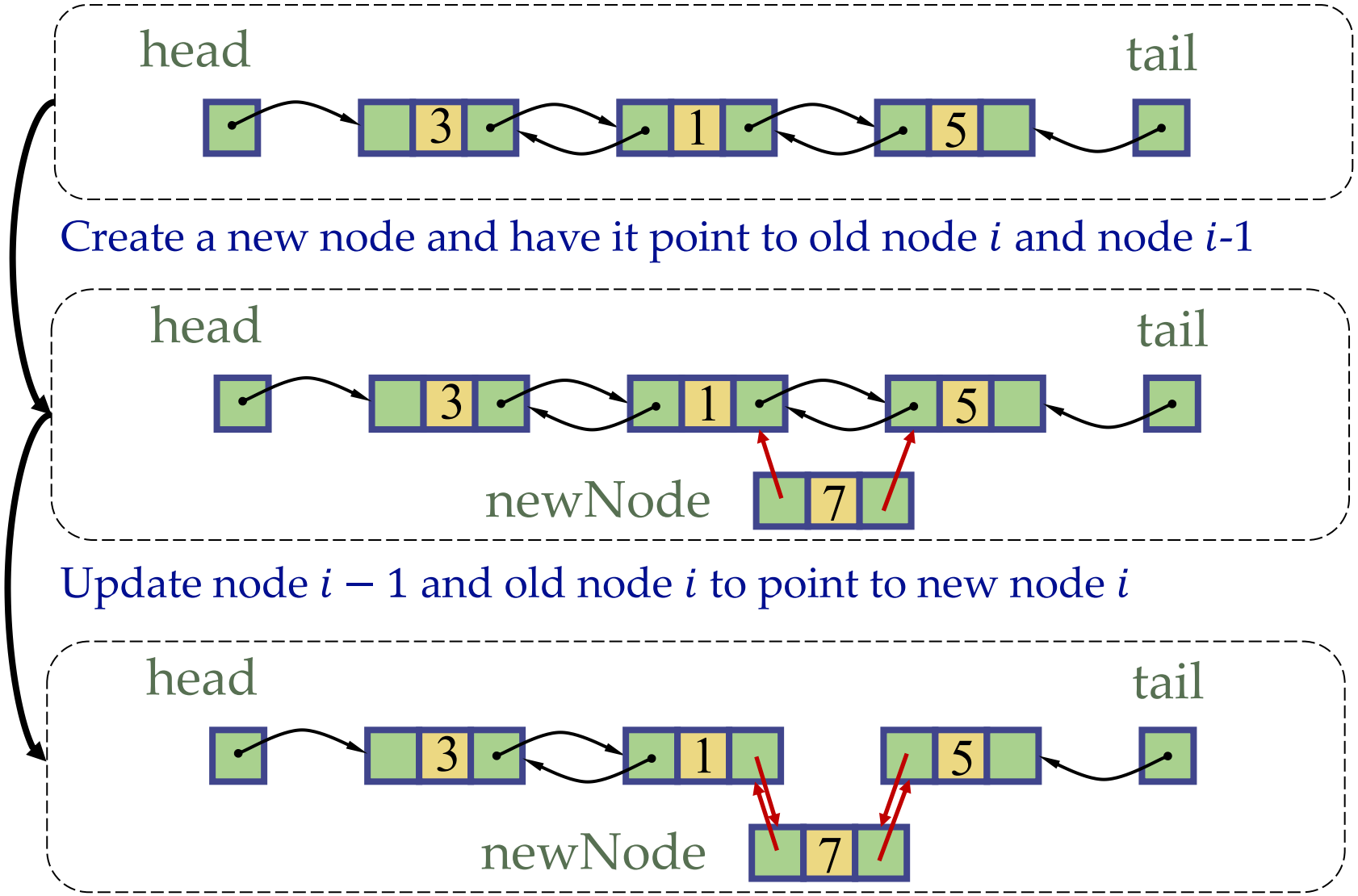

```java
    /** a nested node class */
    public class Node {
        public Node prev;
        public int element;
        public Node next;

        public Node(Node p, int e, Node n) {
            prev = p;
            element = e;
            next = n;
        }
    }
```

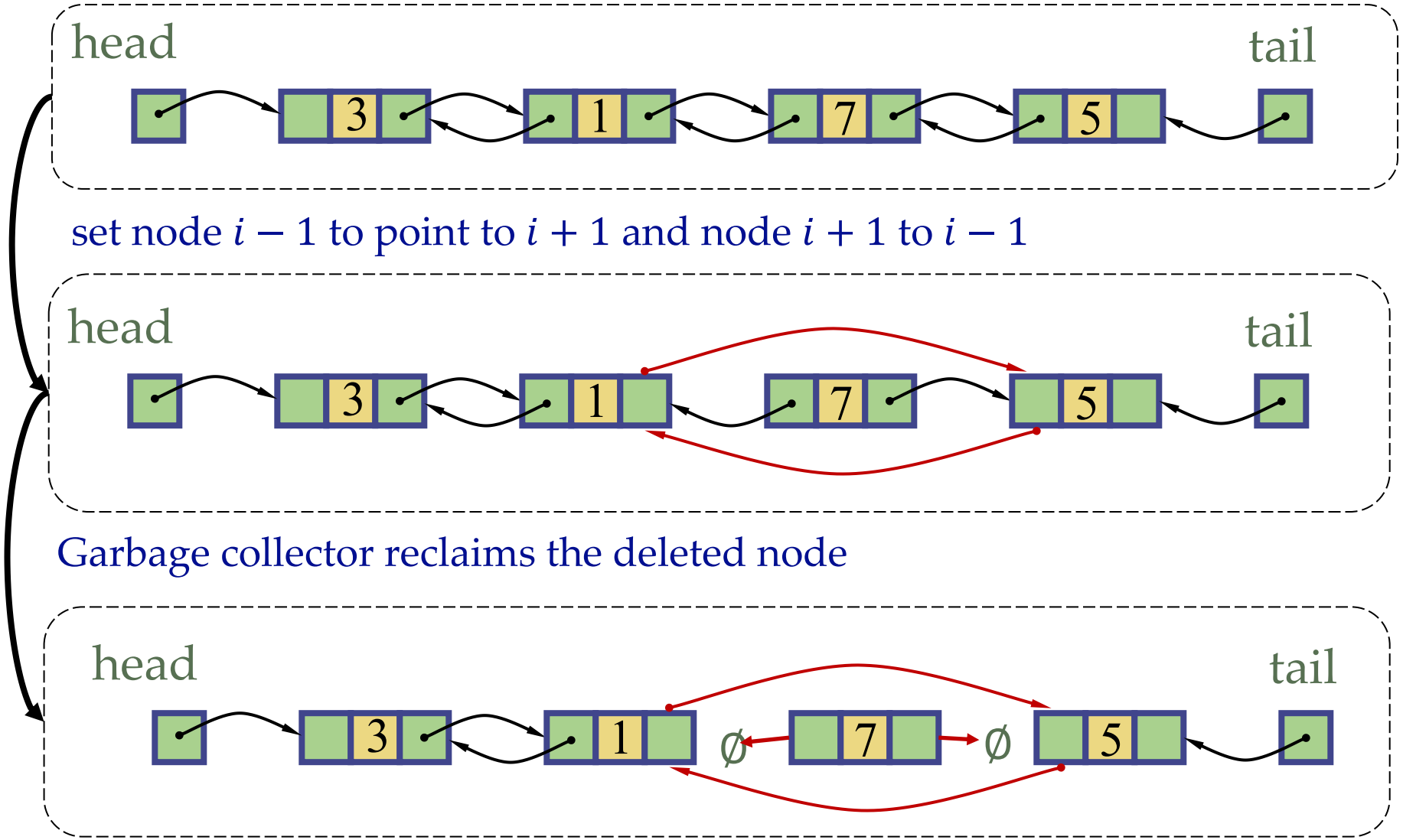
```java
/** create and print the link list */
public static void main(String[] args) {
        DLList list = new DLList(3);
        list.addLast(1);
        list.addLast(7);
        list.addLast(5);

        list.print();
    }
```

Operations on DLList: Inserting an element at node *i*



Operations on DLList: Deleting the element at node *i*



Generic SLList

```
public class SLListG<MyType> {
    /** A nested node class */
    public Node head;

    public SLListG(MyType e) {
        head = new Node(e, null);
    }

    public void addLast(MyType e) {
        Node current = head;
        while(current.next != null) {
            current = current.next;
        }
        current.next = new Node(e, null);
    }

    public void print() {
        Node current  = head;
        while(current != null) {
            System.out.print(current.element);
            current = current.next;
        }
        System.out.println();
    }
    /** create and print my list */
}
```

```
    /** A nested node class */
    public class Node {
        public MyType element;
        public Node next;

        public Node(MyType e, Node n) {
            element = e;
            next = n;
        }
    }
```

```
    public static void main(String[] args) {
        /** create and print an integer list */
        SLListG<Integer> listInt = new SLListG<>(3);
        listInt.addLast(1);
        listInt.addLast(7);
        listInt.addLast(5);
        listInt.print();

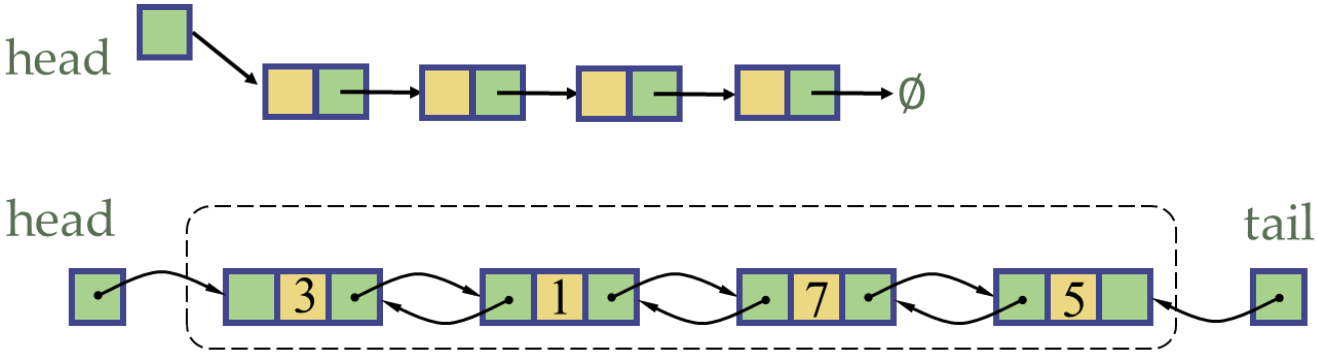
        /** create and print a string list */
        SLListG<String> listStr = new SLListG<>("Hello");
        listStr.addLast("World!");
        listStr.print();
    }
```

Summary of arrays and linked list



Array

- Static structure ☹
- Insertions and deletions require to move elements ☹
- Allow direct access to an element ☺



Linked list

- Dynamic structure ☺
- No need to move elements for insertions and deletions ☺
- No direct access to an element (necessary to traverse the list) ☹